

Creation of Intelligent Bug Diagnosis Platform with Fix Recommendation Assistance

Updated Project Guide - Final Working Version

Project Name: BugSense AI

Frontend: React + Vite | Backend: FastAPI | Knowledge Retrieval: TF-IDF + Cosine Similarity

1. Solution Snapshot

BugSense AI is a developer-focused defect diagnosis platform that accepts bug reports, analyzes them through a five-agent pipeline, reuses historical resolved bugs, detects likely duplicates, and recommends practical fixes.

- Supports both TXT/PDF file upload and direct text input.
- Includes ready-to-use Payment, Login, and Database sample cases.
- Runs Triage, Log Analysis, Root Cause, Duplicate Detection, and Remediation agents.
- Displays severity, priority, component, exception, failure point, root cause, confidence, duplicate status, remediation, prevention, and testing guidance.
- Stores recent analyses in browser history and exposes a functional Dashboard, Knowledge Base, and AI Assistant.

2. Architecture & Technology Choices

Layer	Technology	Purpose
Frontend	React + Vite	Interactive UI, navigation, history, dashboard, assistant
Backend	FastAPI	REST API and diagnostic pipeline
File Parsing	PyPDF2 + text parsing	TXT/PDF bug report extraction
Knowledge Store	historical_bugs.json	Resolved bug knowledge base
Similarity	scikit-learn TF-IDF + cosine similarity	Ranks similar historical bugs
Persistence	localStorage	Stores browser-side analysis history

3. What the Platform Delivers

- Analyze Bug: upload .txt/.pdf reports or paste bug details directly.
- Sample Cases: one-click Payment, Login, and Database examples for demonstration.
- Bug Risk Intelligence: rule-based 0-100 risk score derived from severity, duplicate status, and confidence signals.
- Developer Action Plan: checklist for failure-point review, remediation, regression testing, preventive validation, and historical duplicate review.
- Dashboard: total analyses, critical bugs, duplicates found, average risk, recent analyses, and pipeline status.
- Bug History: searchable and severity-filtered records saved in localStorage.
- Knowledge Base: searchable/filterable display of resolved historical defects with root cause and resolution.
- AI Assistant: interactive follow-up help grounded in the most recent analysis result.

4. Five-Stage Diagnostic Journey

1. Triage Agent

Determines severity, priority, likely component, and reasoning.

2. Log Analysis Agent

Extracts exception type, failure point, affected code path, and stack-trace information.

3. Root Cause Agent

Uses the current bug plus similar historical defects to produce a probable root cause with confidence.

4. Duplicate Detection Agent

Uses similarity-ranked historical bugs to identify likely duplicates and confidence.

5. Remediation Agent

Suggests recommended fix, prevention guidance, testing recommendation, and confidence.

5. Repository Map & Key Locations

- backend/main.py - FastAPI app and endpoints.
- backend/triage_agent.py - severity/priority/component classification.
- backend/log_analysis_agent.py - stack trace and exception parsing.
- backend/root_cause_agent.py - root-cause reasoning from retrieved historical matches.
- backend/duplicate_detection_agent.py - duplicate classification.
- backend/remediation_agent.py - fix/prevention/testing recommendations.
- backend/rag_retriever.py - TF-IDF + cosine similarity retrieval.
- knowledge_base/historical_bugs.json - resolved bug knowledge base.
- frontend/src/App.jsx - main React application.
- frontend/src/App.css - application styling.

6. Environment Readiness Checklist

- Python virtual environment available in the project folder.
- Node.js and npm installed.
- Backend dependencies installed from requirements.txt.
- Frontend dependencies installed from package.json/package-lock.json.
- Do not submit .env files, API keys, passwords, private tokens, venv, node_modules, or __pycache__ folders.

7. Launching the API Service

From the project root in PowerShell:

```
.\venv\Scripts\Activate.ps1  
python -m uvicorn backend.main:app --reload
```

Expected API base URL: <http://127.0.0.1:8000>

Swagger documentation: <http://127.0.0.1:8000/docs>

8. Starting the User Interface

```
cd frontend  
npm run dev
```

Typical development URL: <http://localhost:5173>

9. Verified API Endpoints

- GET / - API health message.
- POST /api/analyze-bug - analyzes uploaded TXT/PDF bug reports.
- POST /api/analyze-text - analyzes directly entered bug text.
- GET /api/knowledge-base - returns resolved historical bugs.

10. Suggested Evaluation Walkthrough

- Open Analyze Bug.
- Choose Enter Bug Details and select Payment Bug.
- Run Intelligent Analysis and review all five agent outputs.
- Open Dashboard and confirm the analysis appears under Recent Analyses.
- Open Bug History and search for Payment.
- Open Knowledge Base and search/filter resolved bugs.
- Open AI Assistant and ask for Root Cause or Recommended Fix.

11. Ready-to-Use Demonstration Scenario

Title: Payment Failure During Checkout

Priority: High

Exception: NullPointerException

File: PaymentService.java

Line: 142

Description:

Customer is unable to complete payment during checkout.

Stack Trace:

```
java.lang.NullPointerException
```

```
at com.shop.payment.PaymentService.processPayment(PaymentService.java:142)
```

```
at com.shop.checkout.CheckoutService.checkout(CheckoutService.java:87)
```

12. What a Successful Demo Should Show

- Triage identifies a high-impact payment/checkout defect.
- Log Analysis identifies NullPointerException and PaymentService.java:142.
- Root Cause Agent retrieves the most relevant historical payment defect.
- Duplicate Detection reports whether the historical similarity is strong enough to classify as a duplicate.
- Remediation recommends initializing/validating the payment object, plus prevention and testing guidance.
- Risk Intelligence calculates a risk score and Developer Action Plan is populated.

13. Similarity Retrieval - Current Final Design

The retrieval layer now uses scikit-learn TF-IDF vectorization and cosine similarity. Historical bug text is built from fields such as title, component, exception, description, root cause, and resolution. The current bug is compared with all historical defects, ranked by similarity, and the top matches are passed to Root Cause and Duplicate Detection.

- Cosine similarity captures textual overlap between the current defect and historical records.
- An exact exception match contributes an additional weighted signal.
- The combined result is scaled to 0-10 to remain compatible with the existing duplicate-detection thresholds.
- This implementation is TF-IDF + cosine similarity; it should not be described as FAISS, ChromaDB, or Sentence Transformer embeddings.

14. Knowledge Reuse Cycle

1. A bug is submitted by file upload or direct text input.
2. The preprocessing and agent pipeline produces structured diagnostic output.
3. TF-IDF + cosine similarity retrieves the most relevant resolved bugs from `historical_bugs.json`.
4. Root Cause and Duplicate Detection use those matches as supporting evidence.
5. Remediation reuses historical resolutions when available.
6. The Knowledge Base page exposes the resolved records to the user for inspection.

15. AI Assistant Behaviour

The AI Assistant is an interactive diagnostic helper grounded in the latest analysis result. It does not require an external API key. It can answer common follow-up questions about root cause, fix, duplicate status, exception, failure point, severity, priority, component, testing, prevention, confidence, and summary.

- Example: 'What is the root cause?'
- Example: 'How do I fix this bug?'
- Example: 'Is this bug a duplicate?'
- Example: 'What should I test?'

16. Functional Screens / Modules

Analyze Bug	Upload + direct text + samples + Clear + 5-agent results
Dashboard	Totals, critical count, duplicate count, average risk, recent analyses, pipeline status
Bug History	Search + severity filter + stored analysis cards
Knowledge Base	Resolved bugs + search + component filter
AI Assistant	Follow-up diagnosis Q&A using latest result
Reports	Simple placeholder unless a reporting page is implemented before submission

17. Troubleshooting

- If React stays on 'Analyzing Bug...', confirm both terminals are running and check the backend terminal for errors.
- Backend terminal: activate the venv and run `python -m uvicorn backend.main:app --reload`.
- Frontend terminal: `cd frontend`, then `npm run dev`.
- If VS Code Problems shows editor warnings but the application runs correctly, they may be linter/interpreter warnings rather than runtime failures; inspect them later if needed.
- If FastAPI imports appear unresolved in VS Code, select the project venv interpreter.
- If port 5173 or 8000 is already in use, stop the older process and restart cleanly.

18. Final requirements.txt

```
fastapi
uvicorn
python-multipart
PyPDF2
nltk
scikit-learn
```

19. Final Submission ZIP - Include / Exclude

Include:

- backend/
- frontend/ source files, package.json, package-lock.json
- knowledge_base/
- bug_preprocessing.py
- requirements.txt
- README.md
- demo_bug.txt if desired

Exclude:

- venv/
- frontend/node_modules/
- __pycache__/
- .env
- API keys, passwords, private tokens
- temporary build/cache files

20. One-Minute Quick Launch

```
Terminal 1: .\venv\Scripts\Activate.ps1
Terminal 1: python -m uvicorn backend.main:app --reload
Terminal 2: cd frontend
Terminal 2: npm run dev
Browser: http://localhost:5173
```